

TVRĐAVA OTPORA

Knezović, Teo

Undergraduate thesis / Završni rad (stručni)

2025

Degree Grantor / Ustanova koja je dodijelila akademski / stručni stupanj: **University of Split, University Department of Professional Studies / Sveučilište u Splitu, Sveučilišni odjel za stručne studije**

Permanent link / Trajna poveznica: <https://um.nsk.hr/um:nbn:hr:228:412090>

Rights / Prava: [Imenovanje 4.0 međunarodna](#) / [Attribution 4.0 International](#)

Download date / Datum preuzimanja: **2026-07-19**

Repository / Repozitorij:



Sveučilišni odjel za stručne studije
SVEUČILIŠTE U SPLITU

[Repository of University Department of Professional Studies / Repozitorij Sveučilišnog odjela za stručne studije Sveučilišta u Splitu](#)



Dabar

SVEUČILIŠTE U SPLITU
SVEUČILIŠNI ODJEL ZA STRUČNE STUDIJE

Stručni prijediplomski studij Računarstvo

TEO KNEZOVIĆ

ZAVRŠNI RAD

TVRĐAVA OTPORA

Split, rujan 2025.

SVEUČILIŠTE U SPLITU
SVEUČILIŠNI ODJEL ZA STRUČNE STUDIJE

Stručni prijediplomski studij Računarstvo

Predmet: Objektno programiranje

Z A V R Š N I R A D

Kandidat: Teo Knezović

Naslov rada: Tvrđava Otpora

Mentor: Ljiljana Despalatović, viši predavač

Split, rujan 2025.

Sadržaj

Sažetak	1
Summary	1
1. Uvod	2
2. Opis korištenih razvojnih alata	3
2.1. Unity	3
2.2. Visual Studio	6
2.3. Aseprite.....	6
3. Opis implementacije elemenata igre	9
3.1. Mape	9
3.1.1. Dizajn mapa	9
3.1.2. Implementacija prepreka i staza.....	12
3.1.3. Učitavanje mapa	14
3.2. Neprijatelji	16
3.2.1. Dizajn neprijatelja.....	16
3.2.2. Struktura i karakteristike.....	18
3.2.3. Valovi	21
3.3. Obrane	22
3.3.1. Dizajn i funkcija obrana.....	22
3.3.2. Implementacija animacija u Unityju	23
3.3.3. Struktura objekta obrana.....	25
3.3.4. Projektili i eksplozije	28
3.3.5. Obrane za generiranje resursa	29
3.3.6. Nadograđivanje.....	29
3.4. Grafičko sučelje.....	31
4. Opis toka igre	34

4.1.	Izbor mape	34
4.2.	Odvijanje igre	35
5.	Zaključak	40
6.	Literatura	41

Sažetak

Ovaj rad se bavi postupkom izrade dvodimenzionalne računalne igre *tower defense* žanra. *Tower defense* je žanr videoigara u kojemu igrač/igračica uobičajeno mora sagraditi obrane (engl. *tower*) da zaustavi postepeno teže napade neprijatelja. Za izradu igre se koristio pogonski sustav Unity, kôd je pisan u programu Visual Studio, a vizualni elementi igre su izrađeni u programu Aseprite.

Cilj ovog rada je izrada interaktivne i zabavne videoigre uz naglasak na intuitivnost i pristupačnost za igrače s raznim razinama vještina te razrada procesa implementacije svih njenih elemenata.

Ključne riječi: Unity, videoigra, *tower defense*.

Summary

Fortress of Resistance

This paper focuses on the process of creating a two-dimensional tower defense game. Tower defense is a genre of videogames in which the player typically needs to construct defenses (towers) to stop gradually stronger waves of enemies. The game was designed using the Unity game engine, the code was written in Visual Studio and the visual elements were designed in Aseprite.

The goal is the creation of a fun and interactive videogame with an emphasis on intuitivity and accessibility for players of all skill levels, as well as detailing the process of implementing all of its elements.

Keywords: Unity, videogame, tower defense.

1. Uvod

Računalne igre su se razvijale dugi niz godina, od arkadnih automata do prijenosnih konzola prošle su kroz mnoge faze napretka te se i dalje kontinuirano razvijaju. Izrada računalne igre, ovisno o vrsti, često zahtijeva vještine iz više područja računarstva i drugih znanosti. Zbog tih razloga se ovaj rad usredotočuje na proces izrade svih elemenata igre te je kao takav dobar pokazatelj kreativnosti, stečenih znanja i vještina.

Danas postoji veliki izbor pogonskih alata za razvoj videoigara (engl. *game engine*) koji znatno olakšavaju proces razvijanja. Za izradu ovog rada se koristio pogonski sustav Unity zato što je vrlo dobro dokumentiran, relativno intuitivan i zbog osobnog iskustva u radu s njim.

Tower defense je samo jedan od mnogih žanrova videoigara te je izabran radi intuitivnosti i pristupačnosti igračima s različitim razinama vještina. Također jednostavnost osnovnog koncepta žanra daje priliku za kreativniju izvedbu ostalih elemenata.

Tvrđava Otpora (engl. *Fortress of Resistance*) je naziv igre, sastoji se od nekoliko elemenata koji uključuju mape na kojima se igra odvija, neprijatelje koji se kreću po stazama navedenih mapa, obrane koje igrači moraju sagraditi da zaustave neprijatelje, grafičko korisničko sučelje i ostali pomoćni elementi.

Rad je podijeljen u šest poglavlja. U sljedećem poglavlju su opisane funkcionalnosti i prednosti razvojnih alata korištenih za izradu igre. U trećem poglavlju je detaljno opisana svrha i implementacija pojedinačnih elemenata igre. U četvrtom poglavlju je dan primjer odvijanja igre kroz kojeg je prikazana interakcija između igrača i elemenata igre. Na kraju je dan zaključak i korištena literatura.

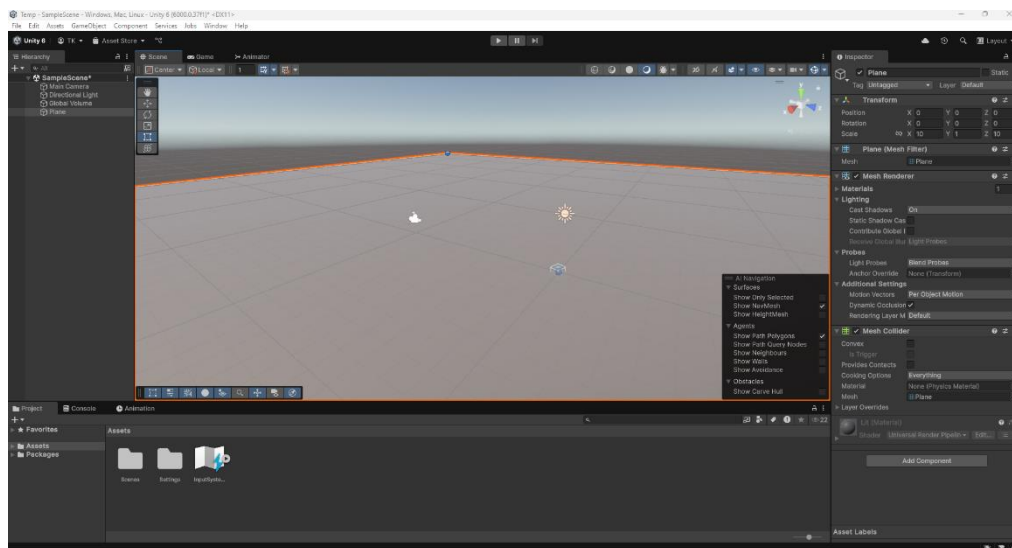
2. Opis korištenih razvojnih alata

U ovom poglavlju su opisane glavne karakteristike i prednosti razvojnih alata koji su se koristili za izradu igre.

2.1. Unity

Kod razvoja računalnih igara izbor *game enginea* je jedan od najvažnijih inicijalnih odluka jer će to težinski diktirati proces razvoja te mogućnost skalabilnosti igre, podrške za različite platforme i generalni opseg potrebnog uloženog vremena za realizaciju različitih elementa igre.

Unity je samo jedan od mnogih pogonskih sustava. Pokazao se kao popularna opcija zbog pristupačnosti i lakoće korištenja, detaljne dokumentacije, ugrađenog sustava za fiziku, animacija, 2D i 3D scena i uz sve to relativno dobrih performansi. Navedeni razlozi ga čine fleksibilnom i dobrom opcijom za bilo koju vrstu projekta, posebno za one manjeg i srednjeg opsega u što spada i Tvrdava Otpora. Na slici 1 je prikazano standardno radno okruženje Unityja.



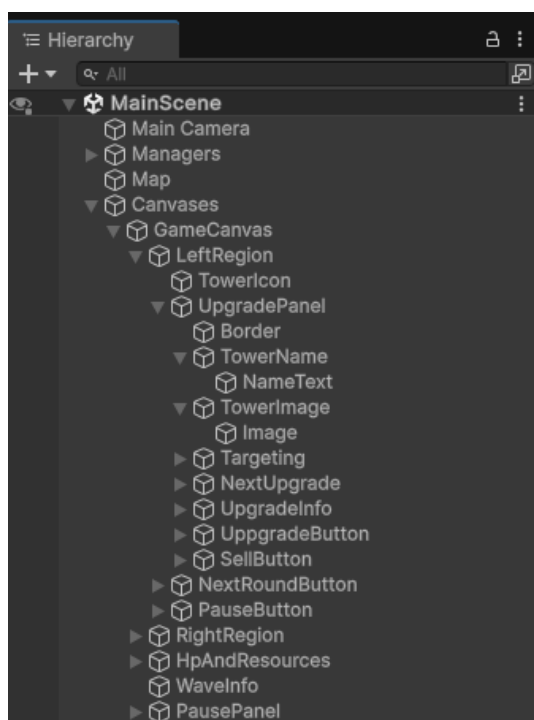
Slika 1: Primjer radne okoline Unityja

Iz slike se primjećuje da radno okruženje sadrži glavni prozor kroz kojeg se prikazuje aktivna scena i elementi u njoj, s lijeve strane se nalazi prozor za hijerarhijski prikaz svih objekta u sceni, s desne strane je prozor za prikaz informacija o izabranom objektu te pri dnu

je smješten prikaz svih direktorija i datoteka aktivnog projekta. Uz navedene postoje i mnogi drugi prozori čija će svrha biti detaljnije opisana u narednim poglavljima uz primjere njihovog korištenja. Također vrijedi naglasiti da su pozicije i veličine svih prozora promjenjive i dinamične.

Glavna karakteristika izrade projekta u Unityju jest da je apsolutno svaki element **objekt igre** (engl. *game object*) koji ima definiranu poziciju i može sadržavati više **komponenti** koje im daju razne funkcionalnosti kao što su prikazivanje geometrijskih tijela, osvijetljavanja, animiranja, interakcije sa sustavom fizike, prikazivanje korisničkog sučelja i mnoge druge.

Objekti igre se nalaze u **hijerarhijskoj** strukturi što znači da pojedini objekti mogu biti djeca (engl. *child object*) drugih. U slučaju da objekt ima pod sobom jedno ili više djece on je njima roditelj (engl. *parent object*). Određene promjene u roditeljskim objektima se mogu odraziti na njihovu djecu, primjerice promjena pozicije roditelja će na identičan način promijeniti poziciju sve njegove djece. Na slici 2 je prikazana hijerarhijska struktura nekoliko objekata.



Slika 2: Prozor hijerarhije objekata u sceni

Svi objekti igre su sadržani u **sceni** što je predefinirana kolekcija objekta. Igra može sadržavati više scena što omogućava brzo učitavanje različitih dijelova igre, tako se izbjegne alternativa gdje su svi elementi igre učitani odjednom što može izazvati znatne probleme kod performanse.

Kao što je prije navedeno svaki objekt može sadržavati različite komponente, među najbitnijim komponentama jest **skripta**. Sa skriptama se mogu postići kompleksna ponašanja objekta, omogućuju im da međusobno komuniciraju te mogu dobiti pristup drugim komponentama i tako dinamički mijenjati karakteristike objekata. Na slici 3 su prikazane komponente objekta među kojima su i dvije skripte.



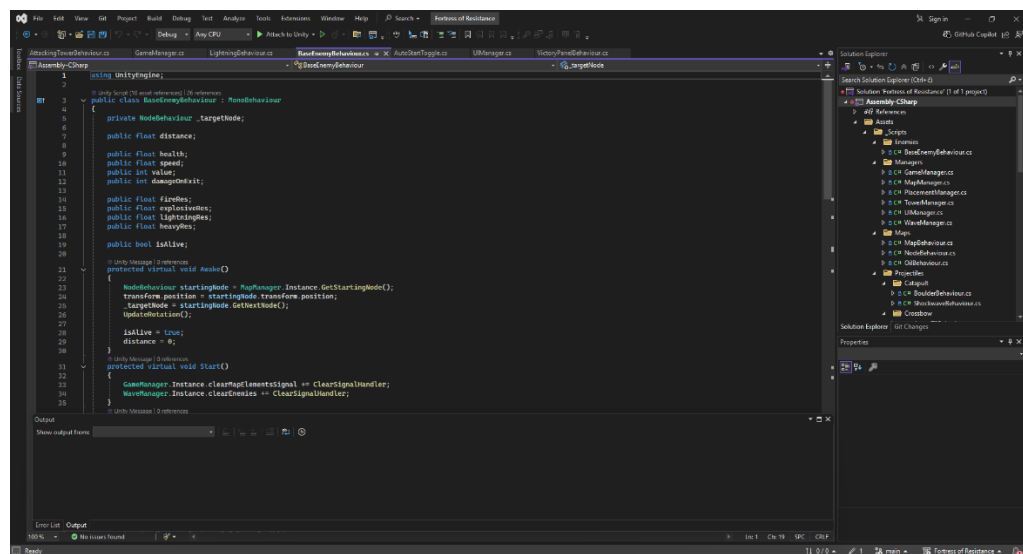
Slika 3: Prikaz komponenti objekta

Unity skripte se pišu u programskom jeziku C#. Pošto je taj jezik isključivo objektno orijentiran jako je intuitivno usklađen s funkcionalnostima objekta igre. Primjerice, objekti igre se vrlo lagano mogu predložiti kao instance klasa u kôdu te njihove komponente kao klase gdje su pojedini parametri komponenti zapravo podatkovni članovi tih klasa i slično.

2.2. Visual Studio

Visual Studio je jedan od najpopularnijih integriranih razvojnih okruženja za pisanje kôda te posjeduje dodatne alate namijenjene za bolju integraciju s Unityjem. Omogućuje laganu i brzu navigaciju kroz sve skripte u projektu i posjeduje kvalitetne alate za pronalaženje i uklanjanje grešaka, isticanje sintakse i sličnih ergonomskih pomoćnika.

Zbog navedenih razloga Visual Studio je bio prirodan odabir uređivača kôda. Na slici 4 je prikazano standardno radno okruženje programa.



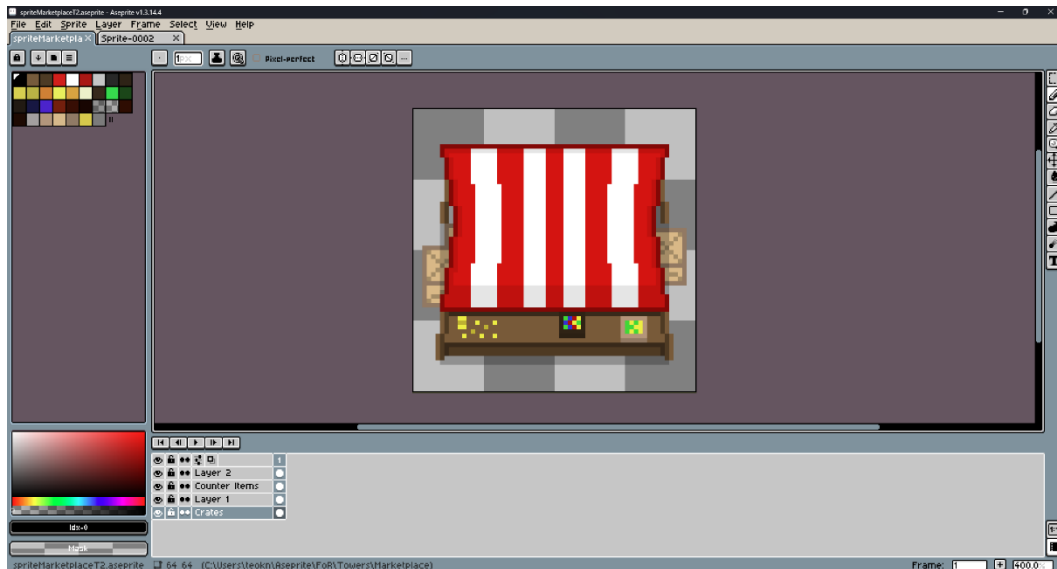
Slika 4: Radno okruženje Visual Studio

2.3. Aseprite

Kod izrade videoigara vizualni dizajn čini ključnu ulogu ne samo u predočavanju pojedinih elemenata igraču, nego i komunikaciju njihove svrhe i funkcionalnosti. Jasno je, dakle, da je izbor vizualnog stila igre vrlo važan korak da bi se moglo izgraditi kvalitetno i intuitivno iskustvo za igrača.

Pošto je Tvrdava otpora dvodimenzionalna igra s perspektivom koja gleda odozgo prema dolje, prirodno je za elemente igre koristiti *spriteove*. Isto tako je bitno da se određeni elementi mogu relativno lagano animirati. Zbog navedenih razloga je korišten **piskelizirani stil** za dizajn elemenata igre.

Aseprite je jedan od mnogih uređivača slika koji se specijaliziraju u izradi pikseliziranih radova (engl. *pixel art*) te je izabran zbog jednostavnosti korištenja i intuitivnog sustava za stvaranje animacija. Na slici 5 je prikazano tipično razvojno okruženje programa.

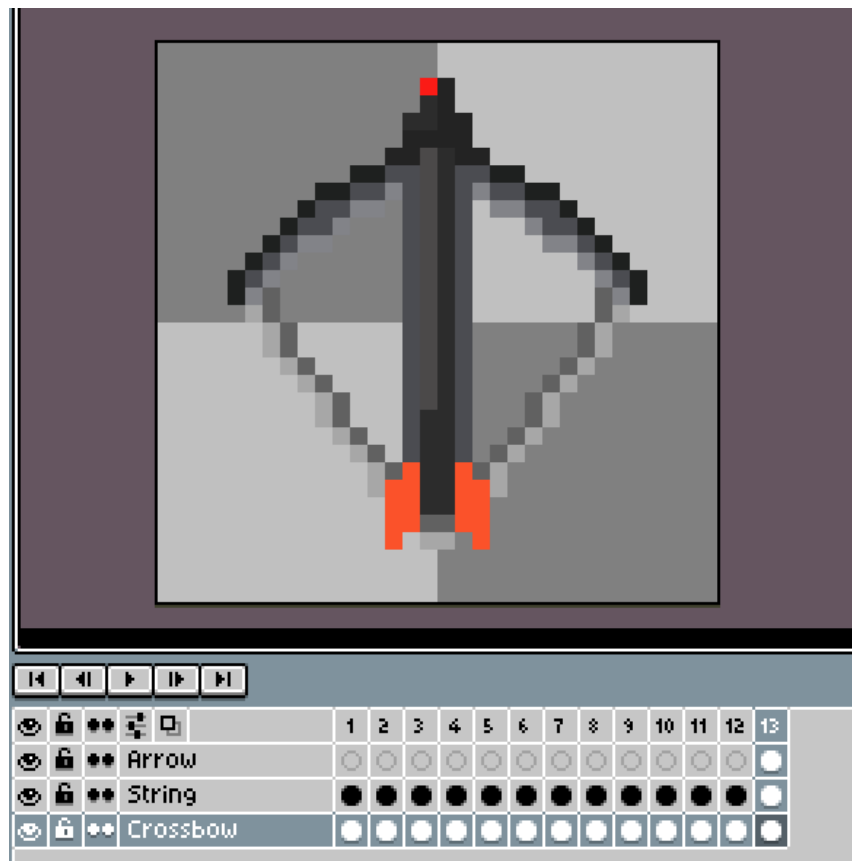


Slika 5: Razvojno okruženje Aseprite

Iz slike se primjećuje da Aseprite posjeduje mnoge standardne elemente koji se tipično nalaze u uređivačima slika kao što su alatna traka (s desne strane), upravljanje bojama (s lijeve strane) te upravljanje slojevima (ispod glavnog prikaza slike).

Uz sve standardne alate prisutne u programu, postoje i dodatni alati koji su specijalizirani za pomoć u izradi pikseliziranih radova. To uključuje prikaz matrice koja pomaže u poravnavanju, prikaz slike koja se ponavlja u vertikalnim i horizontalnim smjerovima (*tiled mode*), alati za simetričnost i slični.

Aseprite također posjeduje sustav za podršku animacija koji su ključni za izradu mnogih elemenata igre. Na slici 6 je prikazan primjer animacijske trake za jednu obranu u Tvrđavi Otpora.



Slika 6: Animacijska traka u Asepriteu

Kao što se može uočiti iz slike, animacije su zapravo niz slika tj. kadrova (engl. *frame*) koje stvaraju prividno kretanje kada se prikazuju uzastopno. U ovom slučaju za animaciju pucanja samostrijela je poredano 13 slika što se može primijetiti po broju stupaca u animacijskoj traci, dok redovi predstavljaju pojedinačne slojeve slike. Svaki od tih slojeva je imenovan i može se individualno uređivati u svakom kadru animacije.

3. Opis implementacije elemenata igre

U ovom poglavlju se detaljno opisuje proces dizajniranja i izrade pojedinačnih elemenata igre.

Bitno je naglasiti da je kod izrade igre nužno da se većina elemenata razvija paralelno zbog njihove međusobne interakcije.

3.1. Mape

U igri tipa *tower defense* mape imaju mnogo važnih uloga. Služe ne samo zbog vizualnih razloga, nego uz to definiraju putanju neprijatelja te s njom i težinu igre. Isto tako različiti elementi na samim mapama mogu diktirati optimalne strategije i pozicije obrana. Primjerice na mapi sa mnogo prepreka je obeshrabreno korištenje većih obrana zbog manjka prostora.

3.1.1. Dizajn mapa

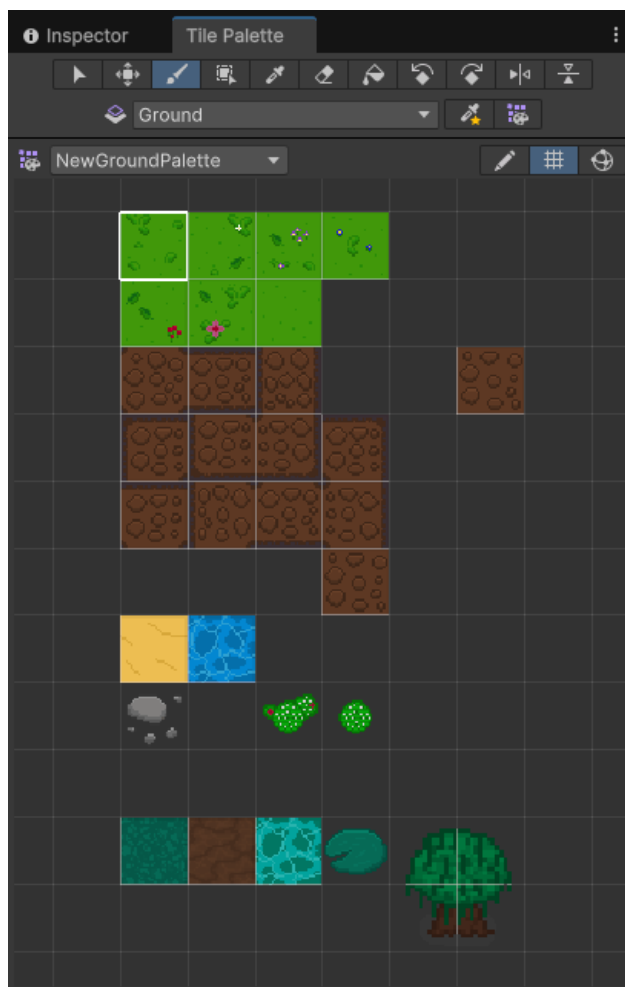
Mape u Tvrđavi Otpora su izrađene kao *tilemapovi*, što znači da su mape efektivno matrice blokova (engl. *tile*) koji su nanizani jedni do drugih. *Tileovi* su nacrtani u rezoluciji **32 x 32 piksela**, njihova visina i širina odgovaraju točno jednoj mjernoj jedinici u Unityju što je važan detalj jer određuje rezoluciju svih ostalih elemenata. Na slici 7 su prikazane varijacije *tilea* trave i staze.



Slika 7: Varijacije *tilea* u Asepriteu

Za korištenje *tilea* u Unityju potrebno je od njihovih slika stvoriti paletu (engl. *tile palette*). Pri uvozu slika potrebno je definirati kolika im je rezolucija naprema jednoj mjernoj jedinici Unityja i s obzirom na to da je riječ o pikseliziranom stilu također ugasiti kompresiju i filtere da se očuva originalni izgled.

Na slici 8 je prikazana glavna paleta za izradu mapa u Tvrđavi otpora koja se nalazi u posebnom prozoru Unityja.



Slika 8: Prozor *Tile Palette*

Paleta omogućuje brzu i jednostavnu izradu mapa, blokovi se odabiru iz nje te se mogu postaviti i izmjenjivati u *tilemapu*. Na sjedećim slikama je prikazan izgled triju mapa u Tvrđavi otpora.



Slika 9: Prikaz mape Livada (nazvane *Meadow* u igri)



Slika 10: Prikaz mape Pustinja (nazvane *Desert* u igri)



Slika 11: Prikaz mape Močvara (nazvane *Swamp* u igri)

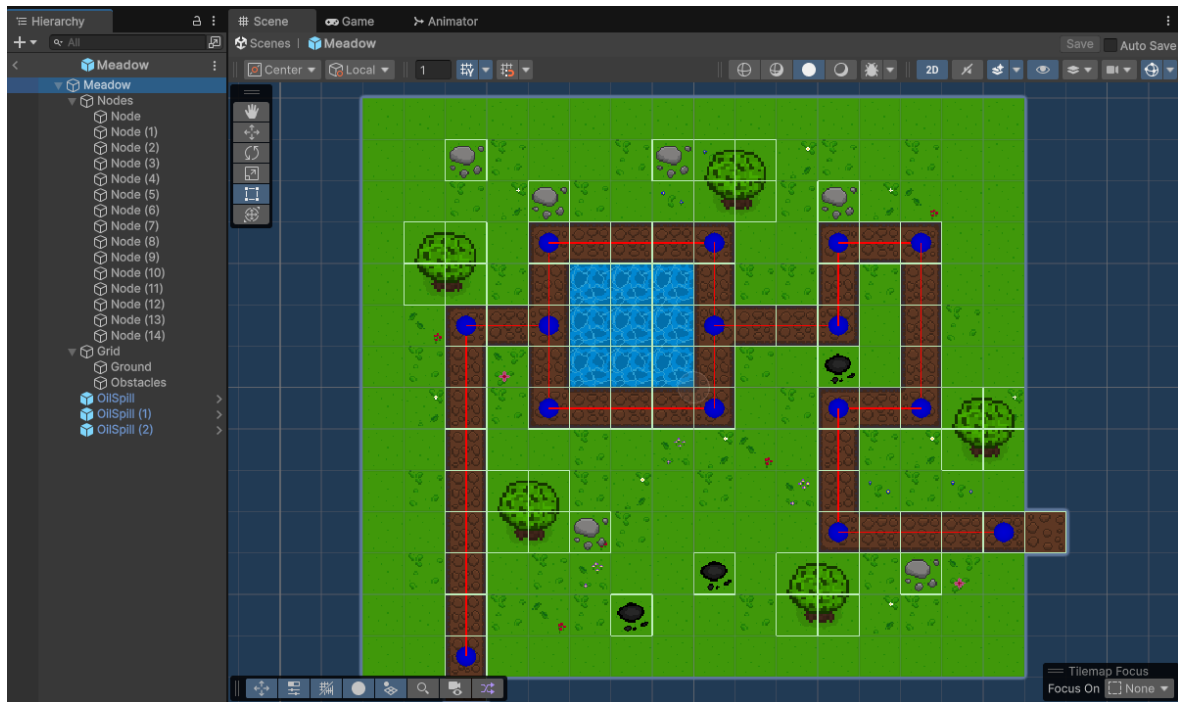
Iz prethodnih slika su očite estetske razlike između mapa, ali isto tako razlike u pozicijama staza i prepreka što je važno zbog strateških razloga u samoj igri pošto se neprijatelji kreću po stazi te se obrane ne mogu graditi na preprekama.

Jedan primjer razlike u samoj težini mapa jest da mapa Livade sadrži dugu stazu s mnogo skretanja i malo prepreka, dok mapa Močvare sadrži dvije potpuno odvojene i vrlo kratke staze te mnogo prepreka kao što su voda i blato u centralnim pozicijama.

3.1.2. Implementacija prepreka i staza

Mape u Tvrđavi Otpora su izrađene od više slojeva blokova, jedan sloj je za blokove na kojih se mogu graditi obrane, dok su ostali slojevi za blokove koji predstavljaju elemente na kojih se ne mogu graditi obrane (kao što su prepreke, staze i voda).

Svi blokovi na kojih se **ne mogu** graditi obrane trebaju imati **collider komponentu**. *Collider* komponente omogućuju objektima detekciju sudara i preklapanja s drugim objektima te im definiraju geometrijski oblik koji definira njihov fizički oblik u kojem detektiraju sudare. Na sljedećoj slici su prikazani svi *collideri* mape Livade.



Slika 12: Prikaz *collidera* svih prepreka na mapi

Na prethodnoj slici je također prikazana hijerarhijska struktura objekta mape. Uz sve *collidere* također se ističu plave točke na stazi što su ustvari čvorišta (engl. *node*) preko kojih je definirano kako se neprijatelji kreću.

Čvorišta na mapi su predefinirane točke izvedene kao vezana lista (svaka točka pokazuje na jednu ili više točaka, što je na slici 12 prikazano kao crvena linija između točaka). Neprijatelji se kreću od početne točke definirane u skripti do sljedeće točke na koju pokazuje te tako sve dok ne dođu do točke koja nema sljedeću i tako znaju da su došli do kraja.

U slučaju da točka pokazuje na više sljedećih točaka, neprijatelji će se od te točke kretati naizmjenice prema pojedinačnim sljedećim točkama te se tako postiže grananje staze. Na ispisu 1 je prikazana skripta `NodeBehaviour` i metoda `GetNextNode` preko koje čvorovi vraćaju sljedeći čvor.

```
public class NodeBehaviour : MonoBehaviour
{
    [SerializeField]
    private List<NodeBehaviour> _nextNodes = new();
}
```

```

private int _nodeIndex = 0;

public NodeBehaviour GetNextNode()
{
    if (_nextNodes.Count == 0)
    {
        return null;
    }
    _nodeIndex = (_nodeIndex + 1) % _nextNodes.Count;
    return _nextNodes[_nodeIndex];
}
}

```

Ispis 1: Skripta za određivanje sljedećeg čvora

Atribut `[SerializeField]` je često korišten u Unity skriptama. Kada se stavi ispred podatkovnog člana omogućuje predefiniranje i praćenje tog podatka direktno iz radne okoline Unityja. U ovom slučaju omogućuje predefiniranje liste sljedećih čvorova (`_nextNodes`).

3.1.3. Učitavanje mapa

Svaka mapa je objekt igre koji je spremljen kao ***prefab*** (skraćeno od engleske riječi *prefabrication*). *Prefab* je predefinirani objekt igre koji se, zajedno sa svim svojim komponentama i djecom, može lagano dinamički stvoriti u sceni iz skripta.

Na ispisu 2 je prikazana metoda za učitavanje različitih mapa koja se nalazi u klasi `MapManager`.

```

public class MapManager : Singleton<MapManager>
{
    [SerializeField] private GameObject _mapParent;
    [SerializeField] private List<GameObject> _maps;

    private GameObject _activeMap;
}

```

```

private int _currentMapIndex;

public void LoadMap(int index)
{
    if (index < 0 || index >= _maps.Count)
    {
        Debug.Log("Index out of range");
        return;
    }

    _currentMapIndex = index;

    if (_activeMap is not null)
    {
        Destroy(_activeMap);
    }

    _activeMap = Instantiate(_maps[_currentMapIndex],
        _mapParent.transform);
}
}

```

Ispis 2: Skripta za učitavanje mapa

Iz skripte se ističu dvije dodatne metode: `Instantiate` i `Destroy`.

Metoda `Instantiate` prima varijablu tipa `GameObject` koja predstavlja *prefab* koji će pri izvršavanju stvoriti u sceni. Također može primiti parametre o poziciji, rotaciji i roditelju koji će poprimiti novi objekt te vraća referencu na njega.

Metoda `Destroy` prima referencu na objekt igre te pri izvršavanju ga izbriše iz scene zajedno sa svom njegovom djecom.

Iz prethodnog ispisa se vidi da skripta sadrži listu svih *prefaba* mapa te ih preko metode `LoadMap` i danog indeksa učitava i po potrebi briše iz scene. Ovaj pristup je pogodan s obzirom na to da su mape vrlo male i ne zahtijevaju mnogo procesorske moći za razliku od pristupa učitavanja novih scena za svaku mapu.

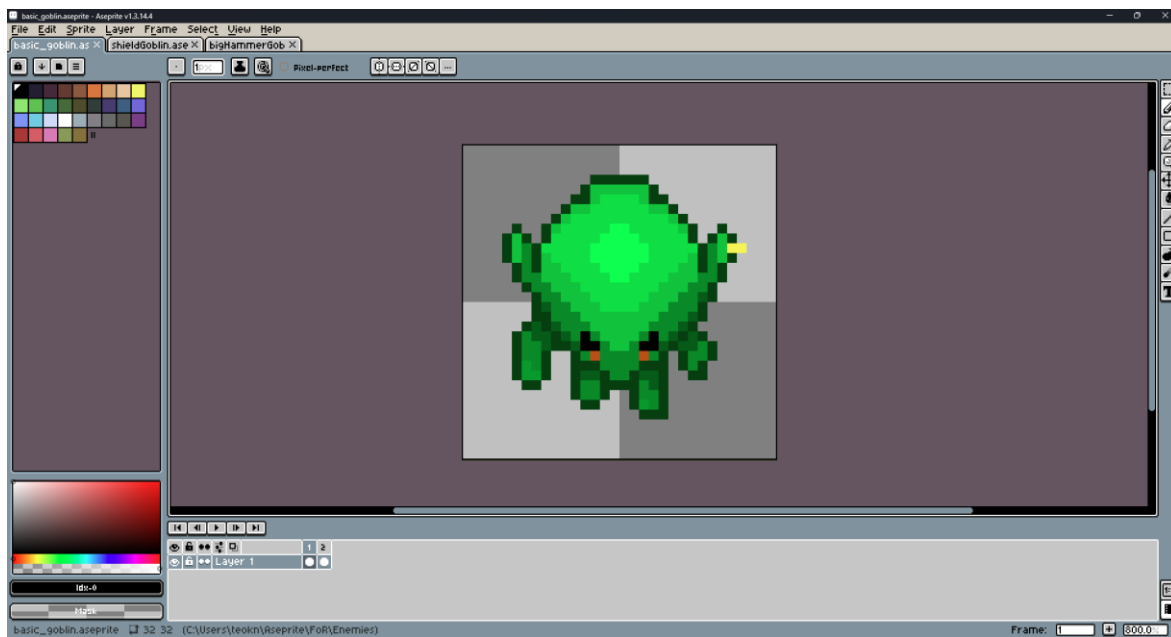
3.2. Neprijatelji

U Tvrđavi Otpora neprijatelji imaju dvije glavne svrhe: služe kao glavna prijetnja igraču zahtijevajući gradnju sve više i više obrana te su glavni izvor resursa za gradnju svih obrana.

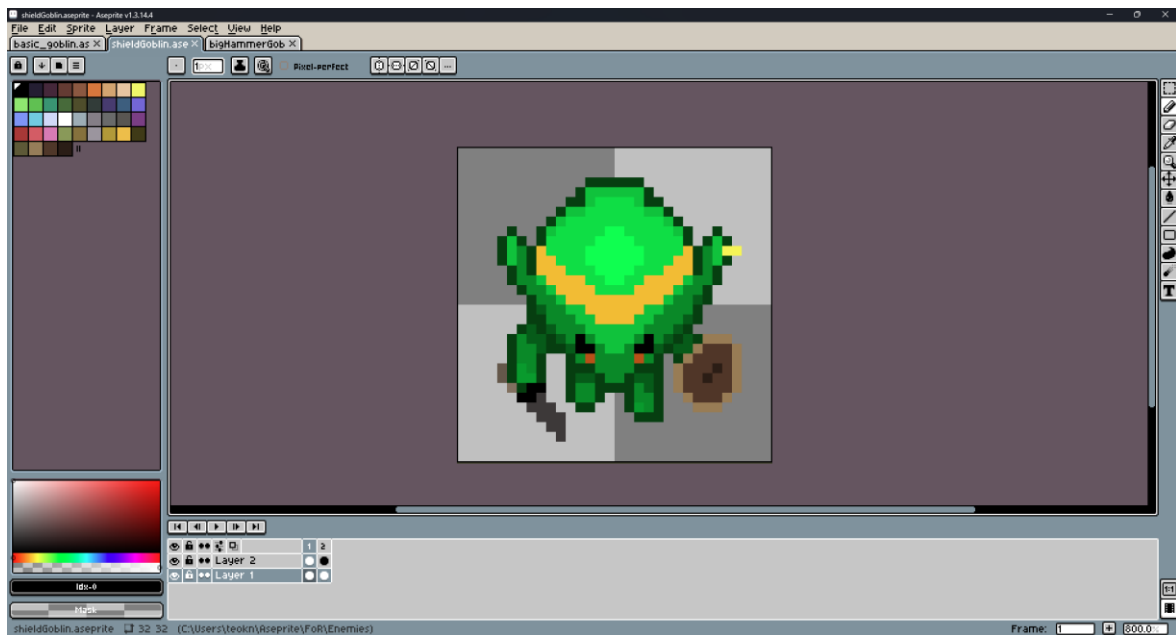
Kao što je u prijašnjem poglavlju navedeno, neprijatelji se kreću od početka staze te je prate sve dok ne dođu do kraja mape pri čemu oduzimaju životne bodove (engl. *health points*) igraču. Ako igrač izgubi sve životne bodove izgubio je igru i mora početi ispočetka.

3.2.1. Dizajn neprijatelja

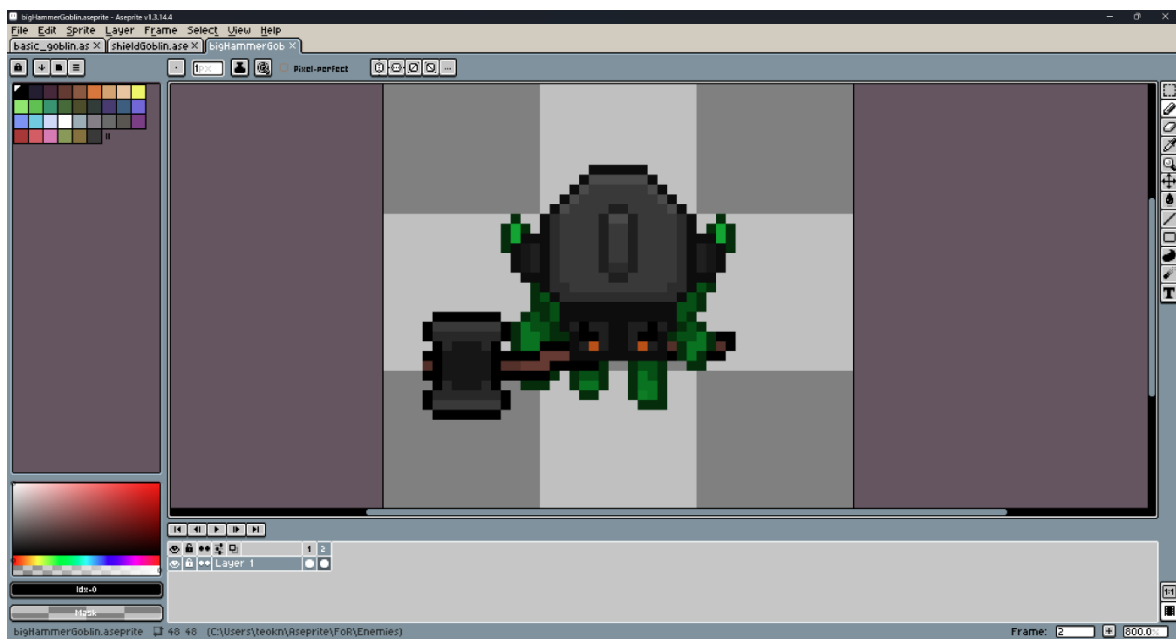
Neprijatelji su dizajnirani kao vrsta *goblina*, u rezoluciji 32 piksela po mjernoj jedinici Unityja, što prati rezoluciju elemenata mapa. Na sljedećim slikama su prikazani dizajni nekih od *goblina* u Asepriteu.



Slika 13: Standardni *goblin*



Slika 14: *Goblin sa mačem i štitom*



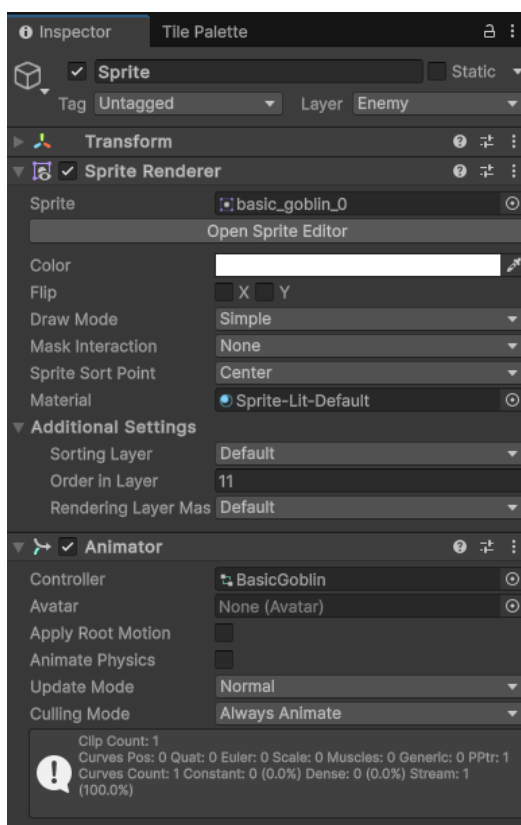
Slika 15: *Goblin sa oklopom i čekićem*

Iz prethodnih slika se primjećuju estetske razlike pojedinačnih neprijatelja. Važno je da igrač može lagano razlikovati pojedinačne neprijatelje te da je jasno prikazano koji od njih predstavljaju veću opasnost.

3.2.2. Struktura i karakteristike

Iz slika neprijatelja je također vidljivo da je svaki od njih animiran. Svaki neprijatelj ima animaciju šetanja koja se kontinuirano odvija dok je živ. Za implementaciju animacije u Unityju potrebno je ugraditi komponentu *animator* koja vodi kontrolu o stanju svih animacija nekog objekta. *Animator* komponenta će se detaljnije opisati u poglavlju obrana gdje služi za mnogo kompleksniju ulogu.

Svaka pojedina vrsta neprijatelja je spremljena kao *prefab*, također svaki objekt neprijatelja posjeduje jedno dijete koje služi kao objekt koji predstavlja *sprite* neprijatelja. Na slici 16 su prikazane komponente koje se nalaze na djetetu objekta neprijatelja.

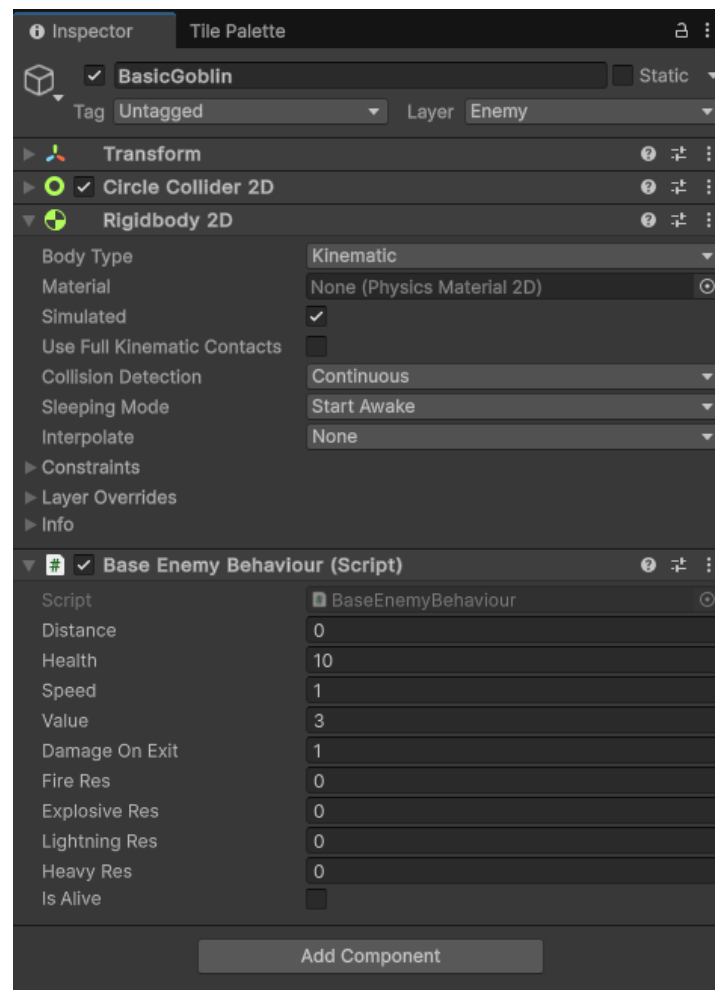


Slika 16: Komponente djeteta neprijatelja

Iz slike je vidljiva prije navedena *animator* komponenta, a također je prisutna komponenta *sprite renderer* koja, kao što se može zaključiti iz imena, je zaslužna za prikazivanje samog *spritea*.

Važno je istaknuti parametar *Order in Layer* u komponenti *sprite renderer* koji služi za odrediti koji *sprite* će se prikazati iznad kojega u slučaju preklapanja. Neprijatelji su poredani tako da se opasniji uvijek prikazuju iznad manje opasnih.

Opasnost neprijatelja je određena prema njihovim karakteristikama koje su vidljive na slici 17 gdje su prikazane komponente objekta neprijatelja.



Slika 17: Komponente neprijatelja

Komponenta *Rigidbody 2D* služi da se objekt klasificira kao takav da na njega utječe sustav fizike Unityja, dok je skripta *BaseEnemyBehaviour* zaslužna za ponašanje neprijatelja.

Kao što je vidljivo iz slike 17, skripta *BaseEnemyBehaviour* sadrži nekoliko prikazanih varijabli koje definiraju karakteristike pojedinih neprijatelja. Važne varijable su *health* (definira koliko životnih bodova imaju neprijatelji), *speed* (definira koliko se brzo

kreću), `value` (definira koliko resursa daje neprijatelj pri smrti), `damageOnExit` (definira koliko životnih bodova oduzima igraču ako dođe do kraja staze) te varijable koje određuju otpore na različite vrste napada.

```
protected virtual void HandleMovement()
{
    transform.position += (_targetNode.transform.position -
transform.position).normalized * speed * Time.deltaTime;

    distance += speed * Time.deltaTime;

    if ((transform.position -
_targetNode.transform.position).magnitude < speed * 2 *
Time.deltaTime)
    {
        transform.position = _targetNode.transform.position;
        _targetNode = _targetNode.GetNextNode();
        UpdateRotation();

        if (_targetNode == null && isAlive)
        {
            isAlive = false;
            GameManager.Instance?.LoseHp(damageOnExit);
            WaveManager.Instance?.OnEnemyDeath();
            Destroy(gameObject);
        }
    }
}
```

Ispis 3: Metoda za kretanje neprijatelja

Na prethodnom ispisu je prikazano kako je kroz skriptu implementirano kretanje neprijatelja. Također je vidljivo kako neprijatelji imaju referencu na ciljanu točku na mapi te u slučaju da su dovoljno blizu njoj pozivaju metodu `GetNextNode` koja je prikazana na ispisu 1.

Posebno je važno istaknuti varijablu `Time.deltaTime` koja je ugrađena u Unity i predstavlja vrijeme koje je prošlo od prethodnog ciklusa ažuriranja (engl. *update cycle* ili *update time*), koristi se u skriptama da osigura da je kretanje elemenata neovisno o performansima igre.

3.2.3. Valovi

U *tower defense* igrama neprijatelji tipično dolaze u postepeno težim **valovima** (engl. *wave*). Tvrđava Otpora koristi predefinirane valove neprijatelja koji su neovisni o mapama, drugim riječima svaka mapa ima iste valove neprijatelja.

Za implementaciju valova u kôdu je stvorena klasa `Group` koja predstavlja grupu neprijatelja iste vrste i prikazana je na ispisu 4.

```
public class Group
{
    public GameObject enemy;
    public int amount;
    public float density;
    public float delay;
    public bool isAsync;

    public Group(GameObject enemy, int amount, float density,
float delay, bool isAsync = false)
    {
        this.enemy = enemy;
        this.amount = amount;
        this.density = density;
        this.delay = delay;
        this.isAsync = isAsync;
    }
}
```

Ispis 4: Klasa koja predstavlja grupu neprijatelja

Iz ispisa se može vidjeti da klasa `Group` sadrži varijable za vrstu neprijatelja, količinu, gustoću (koliko brzo dolaze neprijatelji iz grupe), vremenski razmak do sljedeće grupe (`delay`) te varijablu `isAsync` koja omogućuje dolazak više grupa neprijatelja istovremeno.

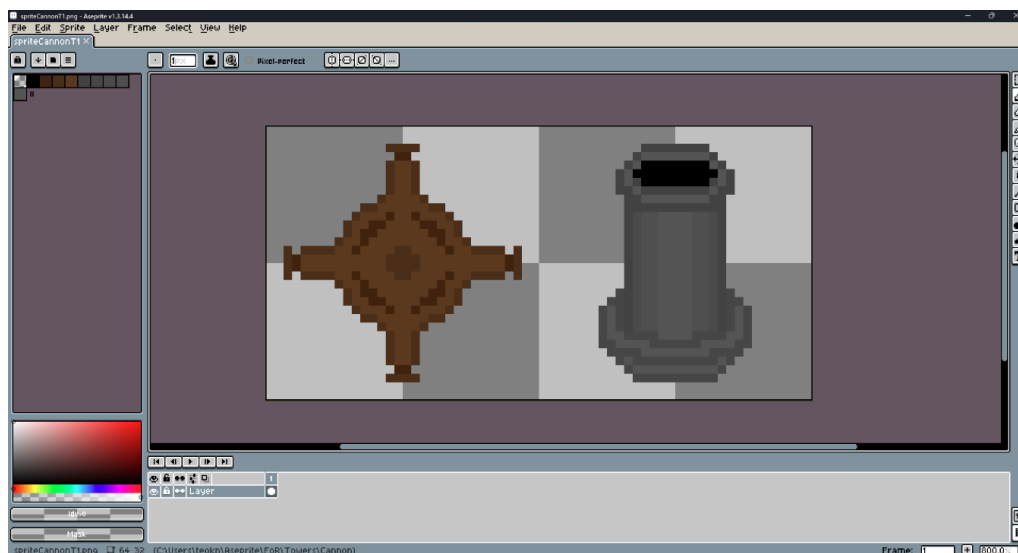
3.3. Obrane

3.3.1. Dizajn i funkcija obrana

U Tvrđavi Otpora obrane su daleko najsloženiji elementi. Razlog tome je što se svaka od ukupno osam ponaša na poseban način te se sve mogu nadograđivati s različitim količinama resursa tako da svaka od tih obrana ima efektivno više verzija s različitim karakteristikama.

Obrane se mogu postaviti samo na blokovima mape, zauzimaju cijeli blok ili u slučaju većih obrana četiri puna bloka te se druge obrane ne mogu postaviti na zauzetim blokovima.

Glavna funkcija obrana jest sprječavanje dolaska neprijatelja do kraja staze, bilo to tako da ih direktno pucaju ili da pridonose dodatne resurse igraču. Postoje dvije vrste, takozvane **napadačke obrane** (engl. *attacking towers*) i **obrane funkcionalnosti** (engl. *utility towers*).

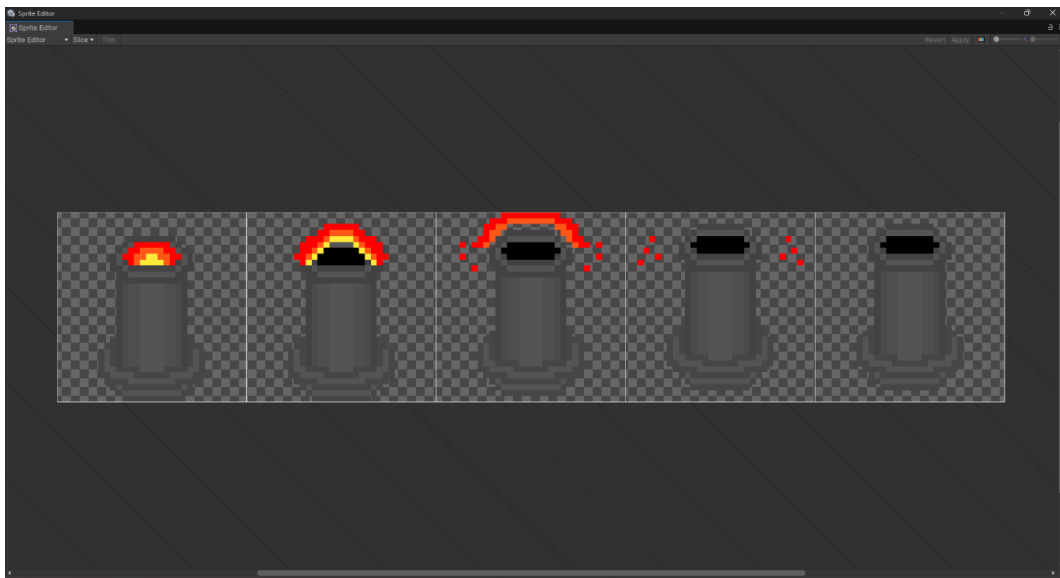


Slika 18: Prikaz dizajna obrane Cannon

Na slici 18 je prikazana najčešća izvedba dizajna obrane, primjećuje se da je slika ustvari podijeljena na dva dijela. Lijevi dio predstavlja **bazu obrane**, dok desni dio predstavlja **glavu obrane**.

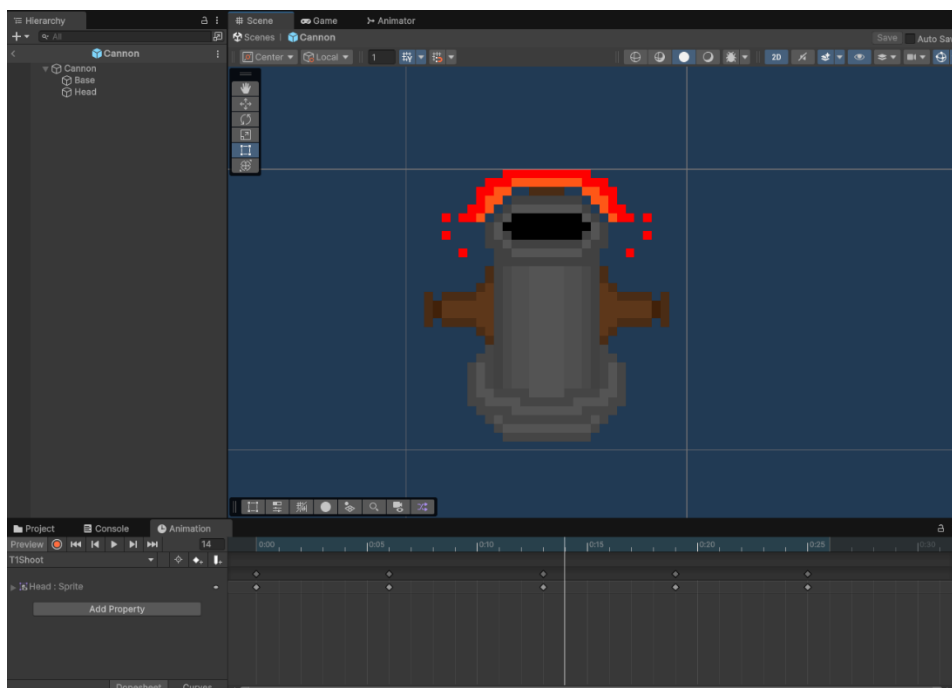
Razlog za podjelu jest to da u igri glava i baza imaju potpuno drugačije funkcije. Glava obrane mora imati mogućnost rotacije te uz to mora imati animaciju za pucanje dok je baza potpuno statična.

3.3.2. Implementacija animacija u Unityju



Slika 19: Prikaz animacije pucanja obrane

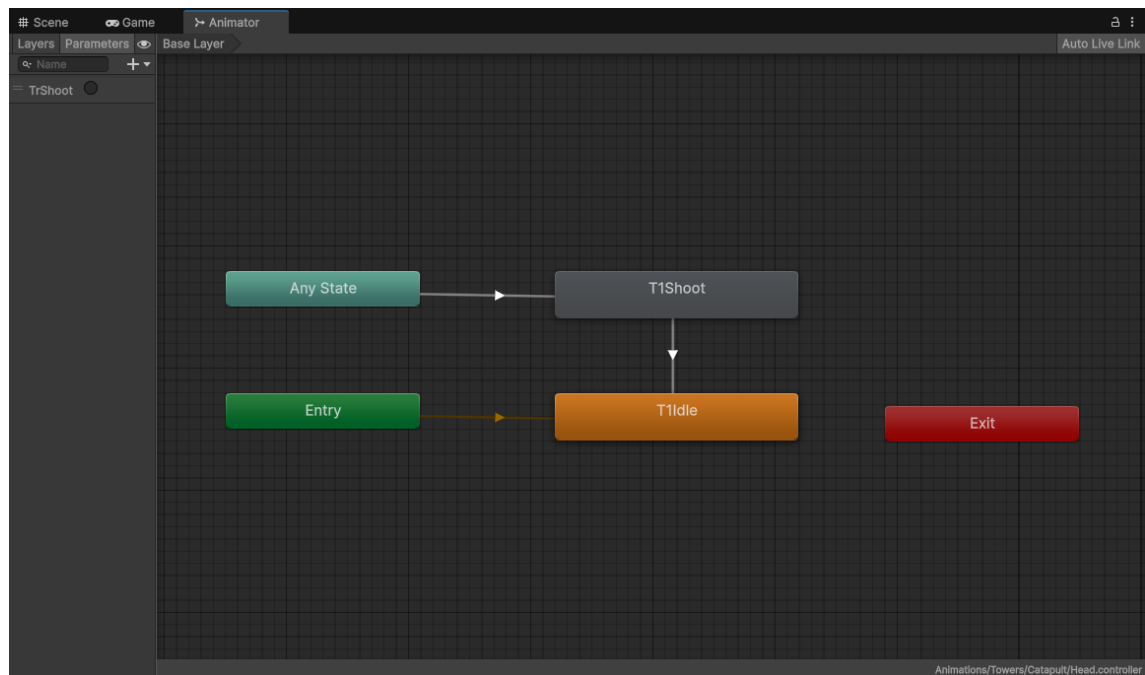
Na slici 19 je istaknuto kako se animacije *spriteova* realiziraju u Unityju. Iz programa Aseprite se niz slika izvozi u traku koja se zatim u Unityju razreže na pojedine kadrove animacije. U ovom primjeru je prikazana animacija od 5 kadrova, no neke od obrana imaju animacije od 15 kadrova i više.



Slika 20: Prikaz animacijske trake u Unityju

Na slici 20 se primjećuju sličnosti između animacijske trake Asepritea i Unityja. Bitno je naglasiti da je traka u Unityju također vremenska traka na kojoj položaj točaka određuje brzinu izvođenja animacije.

Nakon što su sve animacije definirane za pojedini element, potrebno im je postaviti uvjete izvođenja. To se u Unityju radi uz pomoć *animator controllera* što je ustvari automat (engl. *state machine*) koji se podešava u posebnom prozoru prikazanome na slici 21.

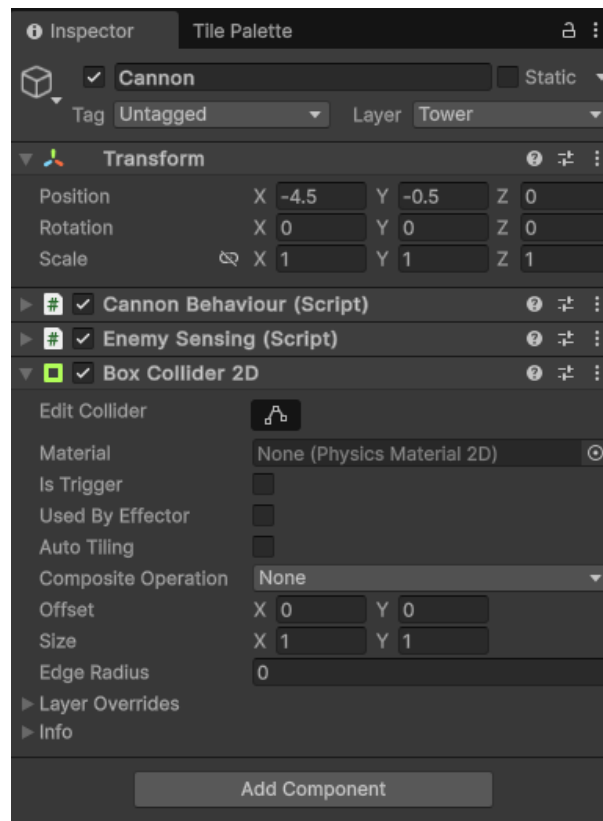


Slika 21: Prikaz prozora Animator

Animacije su na slici predstavljene kao blokovi, dok su uvjeti prelaženja iz jedne animacije (stanja) u drugu predstavljeni strelicama između blokova, svaka od kojih se može konfigurirati.

3.3.3. Struktura objekta obrana

Na slici 22 su prikazane komponente objekta obrane u Unityju, sve obrane su spremljene kao *prefabovi* slično kao i ostali elementi.



Slika 22: Prikaz komponenti obrane

Iz slike se primjećuje da obrane sadrže dvije skripte te *collider* komponentu koja služi za detekciju preklapanja obrana i prepreka (ili drugih obrana) pri postavljanju.

Sve napadačke obrane imaju *EnemySensing* skriptu (prikazana na ispisu 5) koja sadrži metodu za dohvaćanje neprijatelja u dometu obrane.

```
public BaseEnemyBehaviour GetTargetEnemy(TargetPriority
targetPrio)
{
    if (Physics2D.OverlapCircle(transform.position,
_towerBehaviour.range, filter, _enemieColliders) == 0)
    {
        return null;
    }

    var enemies = _enemieColliders.Select((enemy) =>
enemy.GetComponent<BaseEnemyBehaviour>());
```

```

switch (targetPrio)
{
    case TargetPriority.First:
        return enemies.Aggregate(enemies.First(), (a, c)
=> c.distance > a.distance ? c : a);

    case TargetPriority.Last:
        return enemies.Aggregate(enemies.First(), (a, c)
=> c.distance < a.distance ? c : a);

    case TargetPriority.Close:
        return enemies.Aggregate(enemies.First(), (a, c)
=>
            Vector3.Distance(transform.position,
c.transform.position) < Vector3.Distance(transform.position,
a.transform.position) ? c : a);

    default:
        return null;
}
}

```

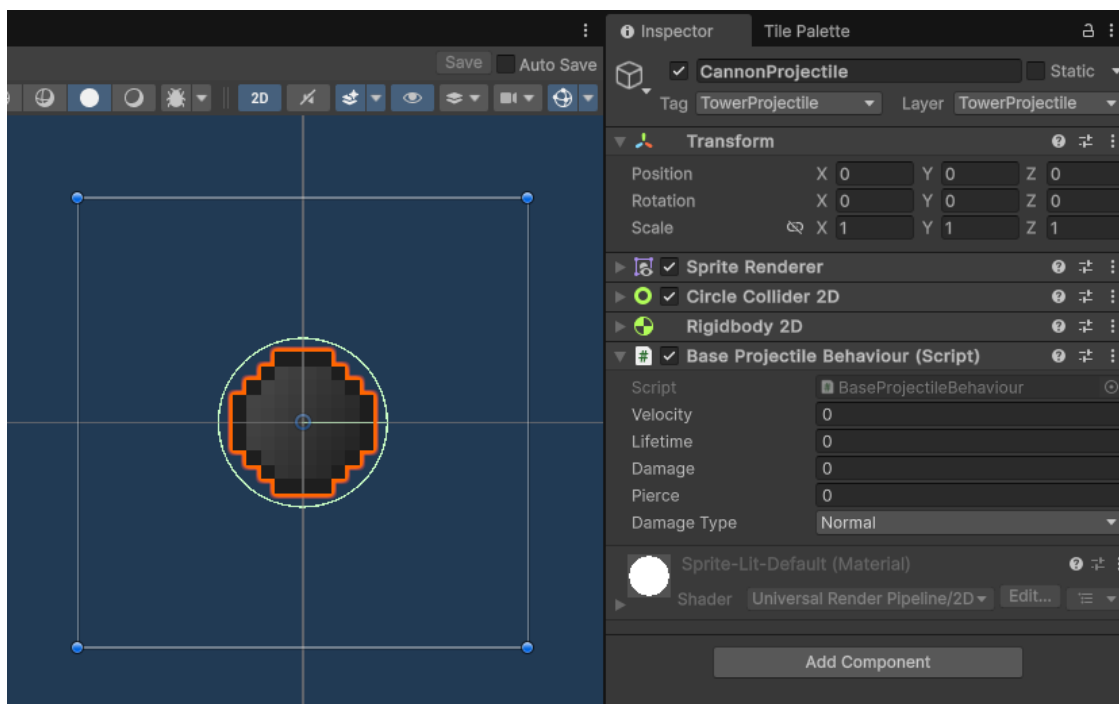
Ispis 5: Metoda za dohvaćanje neprijatelja u dometu obrane

Metoda koristi ugrađenu metodu `Physics2D.OverlapCircle` koja pohranjuje sve filtrirane *collidere* koji se preklapaju u radijusu dometa obrane. Također se primjećuje da postoje **tri vrste prioritiziranja neprijatelja**: `TargetPriority.First` prioritizira neprijatelja najbližem kraju staze, `TargetPriority.Last` neprijatelja najbližem početku staze i `TargetPriority.Close` neprijatelja najbližem samoj obrani.

Sve obrane posjeduju glavnu `Behaviour` skriptu koja je zaslužna za većinu funkcija obrane (pucanje, pridonošenje resursa, nadograđivanje i ostalo), u slučaju iz slike 22 to je skripta `CannonBehaviour`.

3.3.4. Projektili i eksplozije

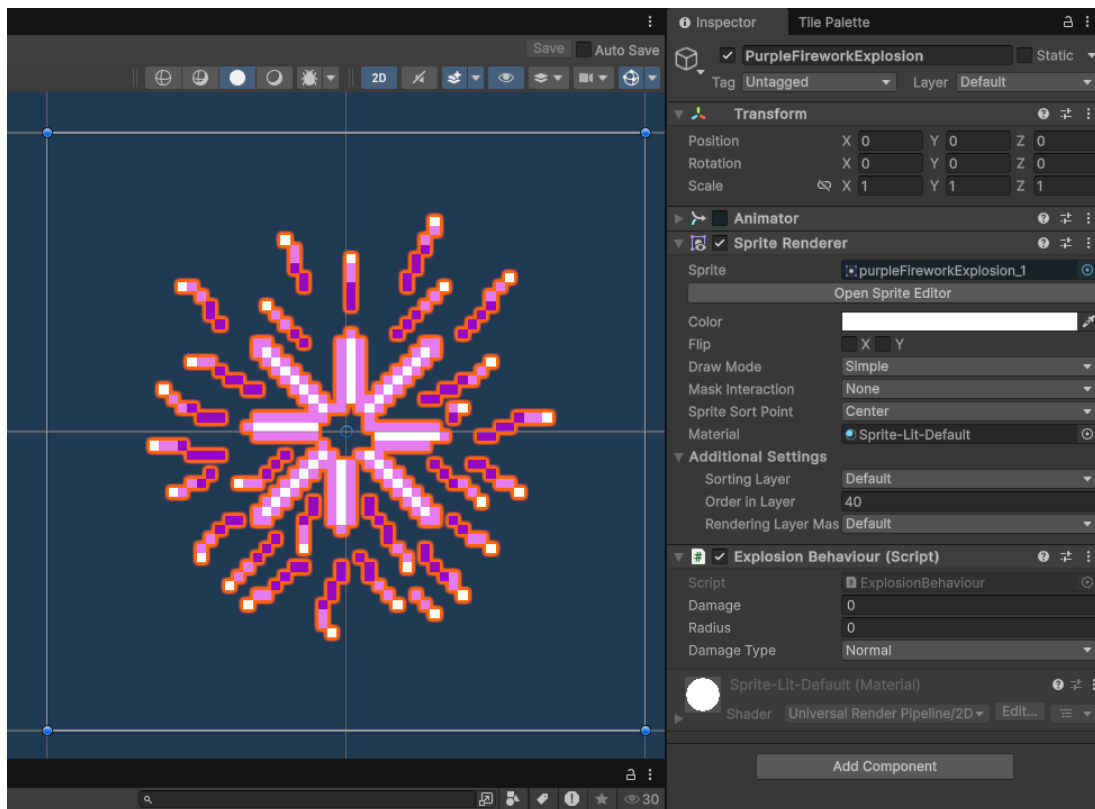
Napadačke obrane prilikom pucanja ispaljuju projekte. Kad se ti projektili sudare s neprijateljem, taj neprijatelj gubi životne bodove. Određeni projektili mogu proći kroz više neprijatelja prije nego što nestanu. Svi projektili su također spremljeni kao *prefabovi* slično kao i ostali elementi. Na sljedećoj slici je prikazan projektil obrane Cannon sa svim komponentama.



Slika 23: Prikaz objekta i komponenti projektila

U skripti `BaseProjectileBehaviour`, koju svi projektili imaju, su istaknute njihove karakteristike preko prikazanih varijabli. Varijabla `velocity` određuje brzinu projektila, `lifetime` maksimalno vrijeme prije nego što se izbriše, `damage` koliko životnih bodova oduzima, `pierce` koliko neprijatelja može udariti prije nego što se izbriše te `damageType` **tip napada** (neki neprijatelji su otporniji na određene tipove napada).

Eksplozije su vrsta napada koje utječu na neprijatelje u određenom radijusu. U Tvrdavi Otpora više obrana ima mogućnost stvaranja eksplozija. Primjer takve obrane je Firework Launcher čiji projektili pri sudaru s neprijateljem stvore eksploziju na mjesto sudara. Na slici 24 je prikazana struktura objekta eksplozije.



Slika 24: Prikaz objekta i komponenti eksplozije

3.3.5. Obrane za generiranje resursa

U Tvrđavi Otpora postoje dvije vrste resursa: **zlato** i **nafta**, koji se koriste za gradnju i nadogradnju obrana. Postoje dvije posebne obrane (*Marketplace* i *Oil Pump*) koje ne napadaju neprijatelje nego služe za generiranje više navedenih resursa. Ovo je također jedini izvor nafte u igri.

Oil pump je posebna obrana koja se jedino može postaviti na izvore nafte koji se nalaze po mapama, svaki izvor ima *collider* komponentu (slično kao i prepreke) koja se nalazi u različitom sloju od prepreka tako da se pri postavljanju obrana može zasebno provjeriti preklapanje s izvorom nafte.

3.3.6. Nadograđivanje

Nakon što se obrane izgrade moguće ih je **nadograđivati**, što zahtjeva dodatne resurse i povećava im razinu (engl. *tier*). Većina obrana se može nadograditi tri puta, kod svake nadogradnje pojačavaju joj se karakteristike (domet, jačina i brzina napada, količina

generiranih resursa i slično) i poprimaju novi izgled. Na ispisu 6 je prikazana metoda koja se poziva pri nadogradnji obrane.

```
public override void UpgradeTier()
{
    switch (tier)
    {
        case 1:
            _damage = 10;
            break;
        case 2:
            range = 3.15f;
            _attackSpeed = 1.45f;
            break;
        case 3:
            _damage = 20;
            _attackSpeed = 0.45f;
            _projectileLifetime = 0.7f;
            break;
        default:
            break;
    }
    base.UpgradeTier();
}
```

Ispis 6: Metoda za nadogradnju obrane

Za promjenu izgleda pri nadogradnji obrane potrebno je dinamički promijeniti izgled baze obrane i u slučaju da je riječ o napadačkoj obrani također promijeniti animacije glave obrane. Za postići to objekti obrana sadrže listu *spriteova* baza te listu ***animator override controllera*** koji mogu dinamički promijeniti set animacija koje objekt koristi.

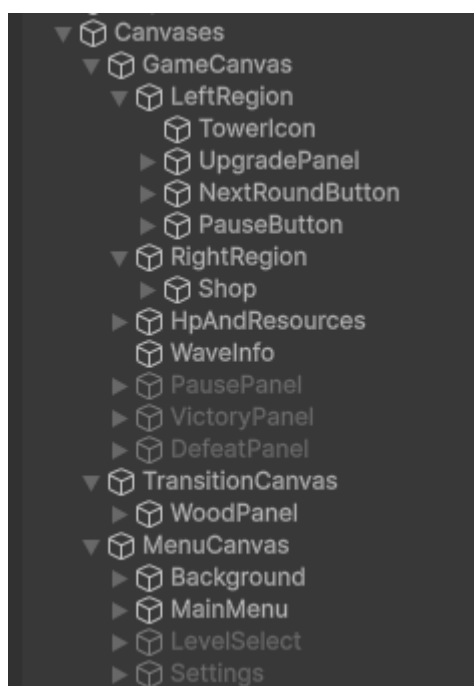
Izgrađene obrane se također mogu **prodati** u bilo kojem trenutku, u tom slučaju igraču je vraćen dio resursa potrošenih na obranu i sve njene nadogradnje. Zbog toga je potrebno

da obrane imaju podatak o ukupnoj količini resursa potrošenih na njima, što se ažurira pri inicijalnoj izgradi i svakoj nadogradnji obrane.

3.4. Grafičko sučelje

U Unityju svi elementi grafičkog sučelja (engl. *GUI elements*) kao što su botuni, pozadine, tekstualni elementi i slično su objekti koji se nalaze u *canvasu*. Canvas je komponenta koja omogućuje prikazivanje i skaliranje elemenata grafičkog sučelja na ekranu.

Pozicija i redoslijed elemenata u hijerarhiji određuje koji će se elementi prikazivati iznad drugih, specifično se djeca elemenata prikazuju iznad roditelja.



Slika 25: Prikaz hijerarhije elemenata grafičkog sučelja

Kao što je vidljivo na slici 25, u Tvrđavi Otpora postoje tri *canvasa*, to su:

1. *GameCanvas* – zaslužan za sve elemente grafičkog sučelja prikazanih tijekom aktivne igre.
2. *TransitionCanvas* – zaslužan za tranziciju između menija i mapa.
3. *MenuCanvas* – zaslužan za glavni meni igre, izbor mapa i postavke.

GameCanvas je podijeljen na lijevu i desnu regiju (prikazano na slici 26). Lijeva regija sadrži panel za nadogradnju obrana i botune za pauziranje igre i početak sljedećeg vala neprijatelja, dok desna regija sadrži panel za gradnju obrana.



Slika 26: Prikaz grafičkog sučelja tijekom igre

Također su prisutni indikatori resursa, životnih bodova i trenutnog vala neprijatelja. Igrač klikom na postavljenu obranu ažurira panel za nadogradnju u kojem su prikazani podaci o odabranoj obrani. Odabrana obrana također dobije prikaz dometa.

Izbornik za izgradnju obrana (*shop*) sadrži botune koji prikazuju cijene obrana. U slučaju da igrač nema dovoljno resursa za gradnju ili nadogradnju obrane odgovarajući botuni postaju neaktivni i prikazuju iznos u crvenoj boji (vidljivo na slici 26).

Botun za početak sljedećeg vala neprijatelja ima dodatnu funkcionalnost modificiranja brzine igre. Tijekom dolaska neprijatelja igrač pritiskom na botun mijenja između normalne i ubrzane brzine igre, naznačeno promjenom izgleda botuna što je prikazano na slikama 27 i 28.

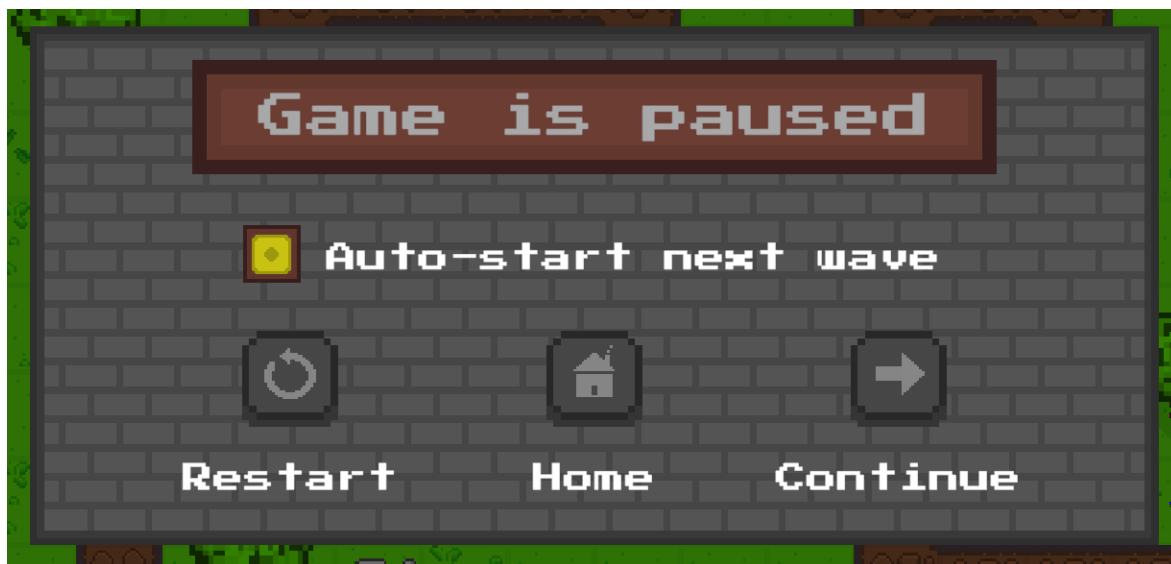


Slika 27: Botun za ubrzavanje igre (normalna brzina aktivna)



Slika 28: Botun za ubrzavanje igre (ubrzana brzina aktivna)

Botun za pauziranje privremeno zaustavlja igru i prikaže panel s botunima za počinjanje ispočetka, izlazak iz mape, nastavljjanje igre te postavku za automatsko pokretanje sljedećeg vala neprijatelja nakon završetka prethodnog. Panel pauze je prikazan na slici 29.



Slika 29: Panel pauze

Ostali elementi grafičkog sučelja, uključujući glavni izbornik, tranzicijski panel i paneli završetka igre su prikazani u poglavlju 4.

4. Opis toka igre

U ovom poglavlju je opisano očekivano iskustvo igrača kroz primjer odvijanja igre. Također su navedene potencijalne strategije koje se mogu primijeniti za veću šansu uspješne obrane.

4.1. Izbor mape

Pri ulasku u aplikaciju igrača dočeka glavni meni (prikazan na slici 30) koji sadrži botune s kojima može započeti igru, promijeniti postavke ili zatvoriti aplikaciju.

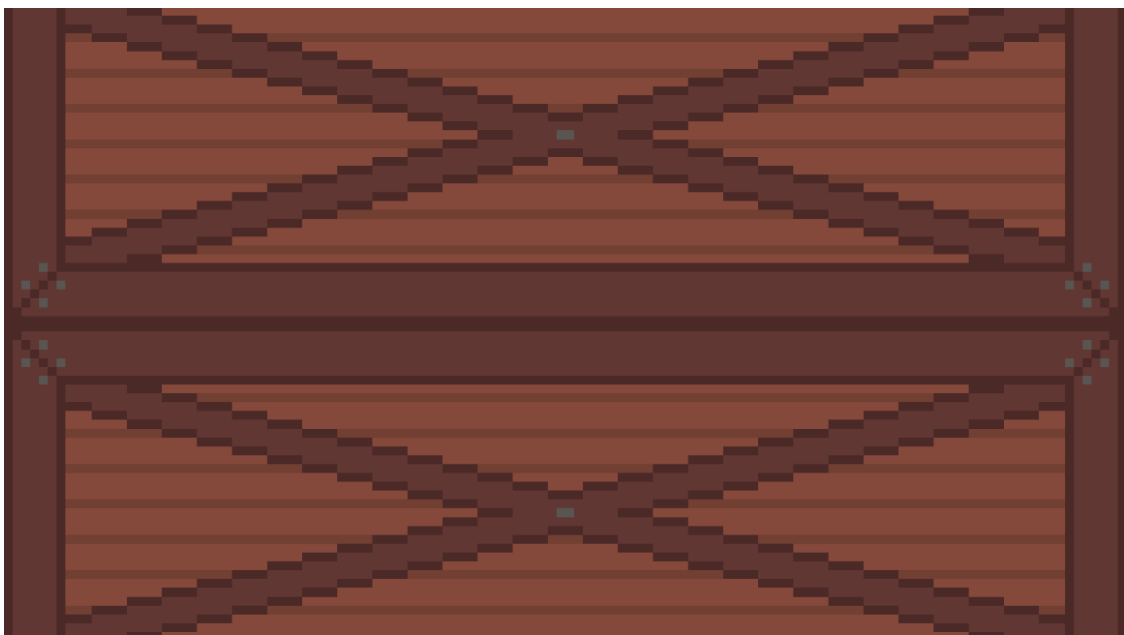


Slika 30: Glavni izbornik

Klikom na „Start Game“ prikaže se izbornik s tri mape poredane po težini, izborom mape odvije se animacija tranzicije s menija ravno u mapu gdje započinje igra. Izbornik mapa i tranzicijski panel su prikazani na slikama 31 i 32.



Slika 31: Izbornik mapa



Slika 32: Tranzicijski panel

4.2. Odvijanje igre

Ulaskom u mapu pustinje igrač prvo ima priliku planirati kako želi postaviti inicijalnu obranu. Za maksimalnu efikasnost obrana važno ih je postaviti tako da imaju što veće pokriće staze, posljedica toga je da pojedine mape imaju različite optimalne pozicije obrana.

Pustinja sadrži dvije odvojene i simetrične staze koje se preklapaju pri sredini mape, to čini centar pogodno mjesto za postavljanje obrana. Važno je napomenuti da se obrane ne mogu postaviti na već zauzeto mjesto tako da igrač mora odlučiti želi li iskoristiti optimalne pozicije za početne slabije obrane ili ih sačuvati za jače obrane nakon što skupi dovoljno resursa. Na slici 33 je prikazana početna postava obrana u ranoj fazi igre.



Slika 33: Primjer početne postave obrana

U Tvrđavi Otpora je nužno investirati dio resursa na obrane za generiranje dodatnih resursa, u protivnom će se jačina neprijatelja povećavati mnogo brže nego što obrane mogu podnijeti.

Ako igrač ulaže resurse u ekonomiju obrana će mu privremeno biti slabija, ako pretjera može vrlo rano izgubiti puno životnih bodova ili čak cijelu igru. Zbog toga je važno **balansirati udio resursa potrošenih na obrane i na ekonomiju** ovisno o stanju igre.

Većinu naprednijih obrana i nadogradnji zahtjeva naftu, koju kao što je prijašnje navedeno je moguće generirati samo iz izvora koji se nalaze po mapama. U pustinji se nalazi ukupno 5 izvora nafte što je daleko najveći broj između svih mapa, što daje mogućnost strategija koji se oslanjaju na skuplje obrane.



Slika 34: Stanje igre u devetom valu

Na prethodnoj slici je prikazano stanje igre nakon što su izgrađene obrane za generiranje resursa. Također je vidljivo da se broj nadolazećih neprijatelja povećao te će uskoro biti potrebna nadogradnja obrana.

Na slici 35 je prikazano stanje igre pri samome kraju gdje su izgrađene neke od najjačih obrana u dok dolazi veliki broj opasnih neprijatelja.



Slika 35: Stanje igre u kasnijim fazama

Primjećuje se da je igraču ostalo samo 25 od početnih 150 životnih bodova, u ovoj fazi to bi značilo da je prolazak jednog neprijatelja dovoljan da izgubi igru. Srećom zbog dobre investicije u ekonomiju trenutna obrana je više nego dovoljna za zaustavljanje svih neprijatelja.

Nakon što igrač zaustavi sve neprijatelje tridesetog vala, igra završava s prozorom koji čestita igraču na uspješnoj obrani (prikazano na slici 36). U slučaju da prije toga izgubi sve životne bodove igra završava s prozorom koji nudi igraču mogućnost ponavljanja igre ispočetka (prikazano na slici 37).



Slika 36: Prikaz prozora pobjede



Slika 37: Prikaz prozora gubitka igre

5. Zaključak

U ovom radu je detaljno prikazan postupak stvaranja videoigre, od dizajna do implementacije i odluka koje su potrebne za kreiranje što boljeg konačnog proizvoda.

Igra je uspješno iskoristila pogodnosti *tower defense* žanra, pokazala se pristupačnom i laganom za razumjeti dok ostavlja opcije za različite strategije i taktike koje igrači mogu primijeniti.

Cijela se igra može iskusiti u malo više od sat vremena. Raznolikost potencijalnih strategija daje razlog igračima da pokušaju riješiti pojedine mape na više načina što može produžiti vrijeme igranja.

Kao kod svake igre postoje mnoga moguća poboljšanja, od dodavanja novih vrsta neprijatelja, mapa i obrana do poboljšanja u području korisničkog sučelja, postavka i podrške za *controllere*. Unity također ostavlja mogućnost da se omogući da korisnici stvaraju vlastite modifikacije igre.

Trenutno bi najvažnija poboljšanja bila dodavanje novih elemenata igre (prvenstveno mapa) u svrhu povećavanja raznolikosti dostupnih strategija i izazova. Također treba osigurati da su jačine različitih obrana primjereno usklađene da se održi raznolikost dostupnih strategija.

Projekt je uspješno stvorio interaktivnu i zabavnu videoigru u kojoj mnogi korisnici mogu uživati.

6. Literatura

- [1] Unity Technologies, „Unity real-time development platform: 3D, 2D, VR & AR engine,“ <https://unity.com/> (posjećeno 27.8.2025.).
- [2] Igar Studio S.A., „Aseprite,“ <https://www.aseprite.org/> (posjećeno 27.8.2025.).